Nombre: José Alejandro Olvera Lugo Fecha: 15/10/2026

Conceptos de Estructuras Dinámicas en Java

1. Definición de Estructuras

Una estructura de datos **dinámica** es aquella cuyo tamaño puede crecer o decrecer durante la ejecución del programa. A diferencia de las estructuras **estáticas**, la memoria se solicita y se libera conforme se necesita.

En Java, cuando queremos crear un nuevo objeto o nodo en tiempo de ejecución, utilizamos la palabra reservada **new.** Este espacio de memoria se asigna en una región llamada **heap.** Las conexiones entre los elementos no dependen de su posición física, sino de las **referencias** que guardan la dirección del siguiente elemento. Cuando un nodo ya no es accesible, el **Garbage Collector (recolector de basura)** se encarga de liberar esa memoria automáticamente.

> **Nota Importante**
> El acceso en una lista dinámica es **secuencial**. Para llegar al elemento n, debemos recorrer todos los anteriores, lo que resulta en una complejidad de *O(n)*.

2. Características Principales

Completa la siguiente lista de características clave de las estructuras dinámicas basándote en la lectura:

- **Tamaño variable:** Se ajusta a la cantidad real de datos en uso.
- **Memoria no contigua:** Los elementos pueden estar dispersos y se conectan mediante enlaces.
- **Eficiencia:** Solo se reserva el espacio estrictamente necesario.
- **Flexibilidad:** La inserción y eliminación no requieren desplazar otros elementos.
- **Costo de memoria:** Cada nodo requiere espacio extra para almacenar sus enlaces.

3. Comparación: Arreglo vs. Lista Enlazada

Analiza las diferencias técnicas entre ambas estructuras y completa los espacios vacíos en la tabla.

Aspecto	Arreglo (Estático)	Lista Enlazada (Dinámica)
Tamaño	Fijo (definido al inicio)	Variable (puede crecer o disminuir)
Acceso por índice	O(1) (Directo)	O(n) (Recorrido secuencial)
Insertar al inicio	O(n) (Requiere desplazar)	O(1) (Solo modifica enlaces)
Uso de Memoria	Bloque contiguo	Memoria no contigua, con enlaces/referencias

4. Análisis de Implementación

¿Por qué es crítico el orden al reasignar referencias durante una inserción? Explica qué sucedería si desconectas el nodo anterior antes de conectar el nuevo nodo a la lista.

Es crítico respetar el orden porque las referencias son las que mantienen conectados los nodos de una lista enlazada.

Si desconectamos el nodo anterior antes de conectar el nuevo nodo, se puede perder la referencia al resto de la lista. Como consecuencia, los nodos que estaban después pueden quedar inaccesibles y perderse durante la ejecución del programa.

El orden correcto es:

1. Crear el nuevo nodo.
2. Conectar el nuevo nodo con el siguiente.
3. Cambiar la referencia del nodo anterior para que apunte al nuevo nodo.

5. Si una estructura requiere recorrer todos los elementos anteriores para encontrar el dato buscado, ¿cuál es su complejidad temporal?

A) O(log n)
B) O(1)
C) O(n)

Nombre: José Alejandro Olvera Lugo Fecha: 15/10/2026

Diagramación de Referencias y Nodos

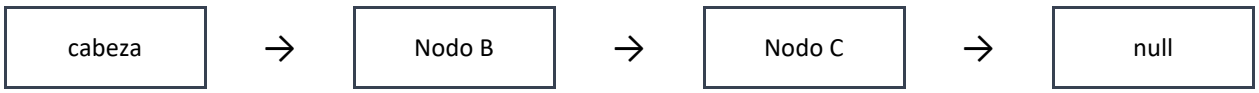
> **Regla de Oro**
> Para no perder el acceso al resto de la lista, **primero conecta el nuevo nodo** a la secuencia existente y **después actualiza la referencia previa**. Si desconectas antes de asegurar el enlace, el Garbage Collector podría eliminar los nodos huérfanos.

En una lista enlazada simple, cada nodo tiene un valor y una referencia al siguiente nodo (next). A continuación, realizarás el trazado visual de cómo cambian estas referencias durante operaciones comunes.

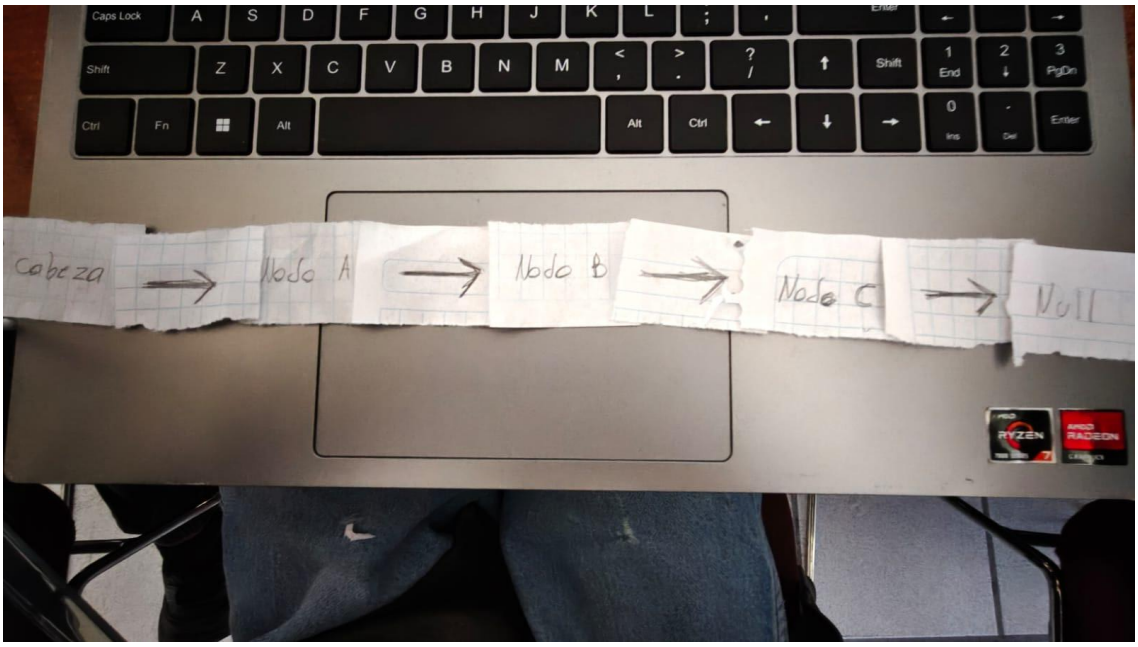
Escenario 1: Insertar al inicio

Se tiene una lista con los valores [B] -> [C]. Se desea insertar un nuevo nodo [A] al principio de la lista. Dibuja cómo deben quedar las flechas de referencia.

Estado inicial (antes)



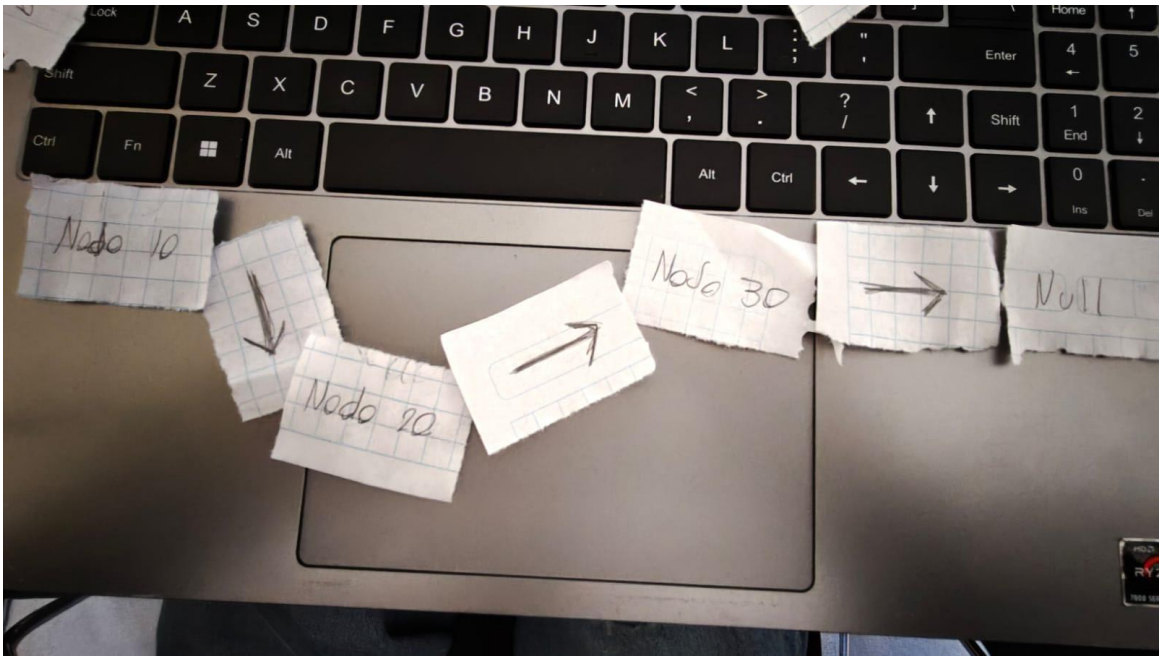
Tu diagrama (después)



Escenario 2: Insertar entre dos nodos

Se tiene la lista [10] -> [30]. Se desea insertar el nodo [20] entre el 10 y el 30. Muestra el orden correcto de las conexiones para no perder el nodo 30.

Estado inicial (antes)
Nodo 10 → Nodo 30 → null

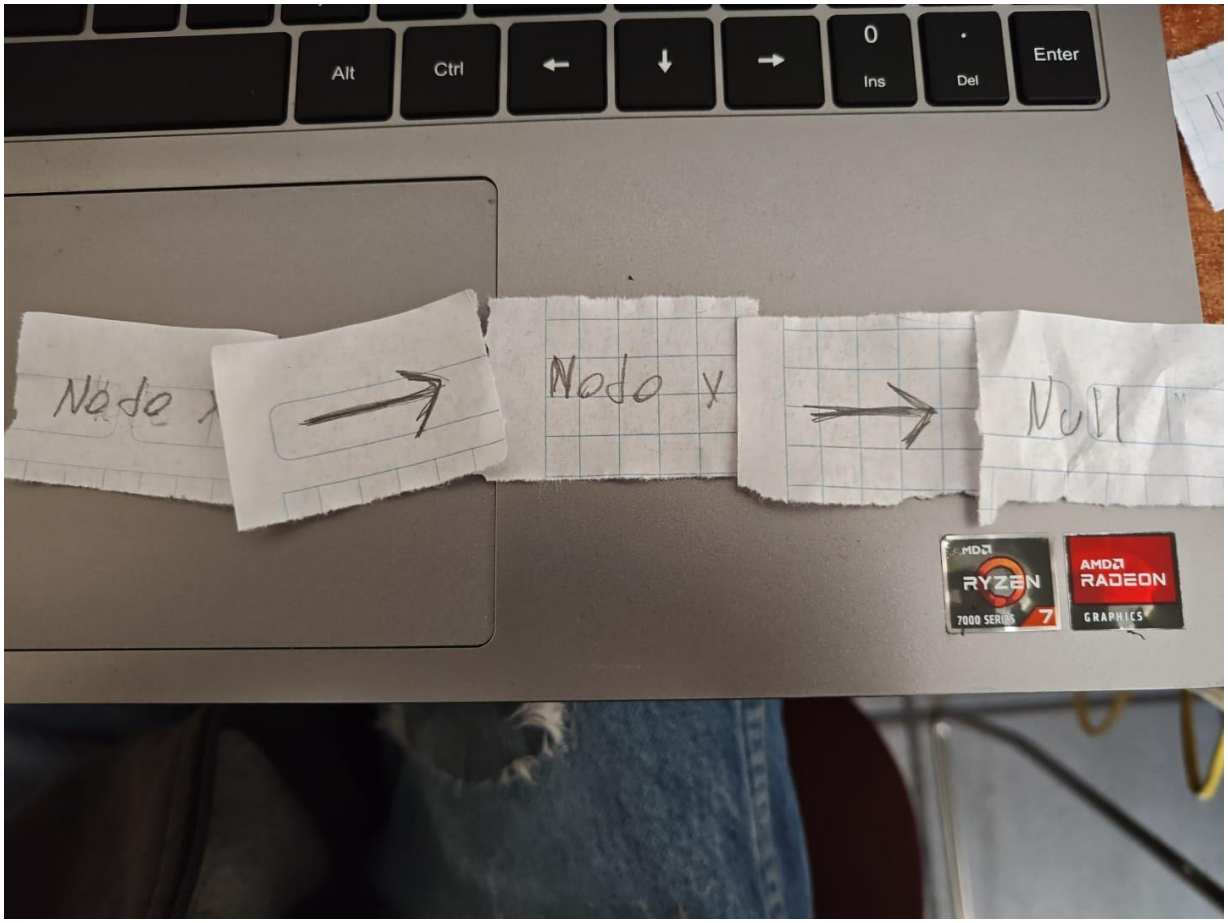
Tu diagrama (después)


Escenario 3: Eliminar el último nodo

Se tiene la lista [X] -> [Y] -> [Z]. Se desea eliminar el nodo [Z]. Dibuja qué referencia debe cambiar a null y qué sucede con el puntero del nodo penúltimo.

Estado inicial (antes)							
Nodo X	→	Nodo Y	→	Nodo Z	→	null	

Tu diagrama (después)



Pregunta de Reflexión:

¿Por qué en el Escenario 2 es vital conectar [20].next = [30] antes de hacer [10].next = [20]?

Si primero hago 10.next = 20, pierdo la única referencia que tenía al nodo 30, porque solo el 10 lo apuntaba. Después ya no hay forma de que el 20 llegue al 30 y la parte restante de la lista se pierde. Por eso primero conecto 20.next = 30 y luego enlazo el 10 con el 20.

Nombre: José Alejandro Olvera Lugo Fecha: 15/10/2026

Laboratorio de Programación: Implementación de una Lista Enlazada

En esta actividad, aplicarás tus conocimientos sobre estructuras dinámicas para construir una lista enlazada simple en Java. Deberás completar el código siguiendo los comentarios TODO. Recuerda que en una estructura dinámica, la memoria se gestiona mediante referencias en el *heap*.

Recordatorio de Lógica

1. **Nodo**: Es el bloque básico que contiene el dato y el enlace al siguiente.
2. **Referencia**: En Java, usamos variables de objeto para apuntar a la dirección de memoria del siguiente nodo.
3. **Orden de Inserción**: Para no perder la lista, primero conecta tu nuevo nodo a la cabeza actual y luego actualiza la cabeza.

Parte 1: La Clase Nodo

Completa el constructor para inicializar el nodo con un valor entero.

```
/**
 * Clase que representa un nodo de la lista simple.
 * Cada nodo guarda un numero entero y la referencia al nodo que sigue.
 * @author Jose Alejandro Olvera Lugo
 * @date 15 octubre 2026
 */

public class Nodo {
	private int info;
	private Nodo proximo;

	// Crea un nodo con el numero recibido y sin nodo siguiente
	public Nodo(int numero) {
		this.info = numero;
		this.proximo = null;
	}

	// Regresa el numero guardado en el nodo
	public int getValor() {
		return info;
	}
```

```
	// Cambia el numero guardado en el nodo
	public void setValor(int info) {
		this.info = info;
	}

	// Regresa el nodo al que apunta este nodo
	public Nodo getSig() {
		return proximo;
	}

	// Cambia el nodo al que apunta este nodo
	public void setSig(Nodo proximo) {
		this.proximo = proximo;
	}
}
```

Parte 2: La Clase ListaSimple

Implementa los métodos para gestionar la estructura. Presta especial atención al método insertarAlInicio para asegurar que la complejidad sea O(1).

```
package Listas.simples;

/**
 * Lista enlazada simple de numeros enteros.
 * Permite insertar al inicio y recorrer la lista imprimiendo sus valores.
 * @author Jose Alejandro Olvera Lugo
 * @date 15 octubre 2026
 */

public class ListaSimple {
	private Nodo primero;

	// Al crear la lista, esta empieza vacia
	public ListaSimple() {
		this.primero = null;
	}

	// Agrega un nodo nuevo al principio de la lista
	public void insertarInicio(int dato) {
```

```
                if (primero == null) {
                        // Si la lista esta vacia, el nodo nuevo es el primero
                        Nodo nodoNuevo = new Nodo(dato);
                        primero = nodoNuevo;
                } else {
                        // Primero enlazo el nodo nuevo con el primero actual y luego lo muevo
                        Nodo nodoNuevo = new Nodo(dato);
                        nodoNuevo.setSig(primero);
                        primero = nodoNuevo;
                }
        }

        // Imprime todos los valores de la lista, de principio a fin
        public void recorrer() {
                Nodo actual = primero;
                while (actual != null) {
                        System.out.print("valor " + actual.getValor() + " -> ");
                        actual = actual.getSig();
                }
                System.out.println("null");
        }

        // Prueba rapida de la lista
        public static void main(String[] args) {
                ListaSimple miLista = new ListaSimple();
                miLista.insertarInicio(2);
                miLista.insertarInicio(3);
                miLista.recorrer();
        }
}
```

Parte 3: Análisis de Casos Límite

Una vez completado el código, analiza cómo se comportaría tu implementación en las siguientes situaciones.

Caso Límite	¿Qué sucede en tu código?
La lista está vacía (cabeza == null)	Al insertar, nuevo.siguiente queda en null y la cabeza pasa a ser el nuevo nodo. Si llamo a recorrer(), el while no se ejecuta y no imprime nada.

Caso Límite	¿Qué sucede en tu código?
La lista tiene un solo elemento	La cabeza apunta a un nodo cuyo siguiente es null. recorrer() imprime ese dato y termina.
Insertar un nuevo nodo cuando ya existen otros	El nuevo nodo apunta a la antigua cabeza y luego se vuelve la cabeza. No se pierde ningún nodo y la operación es O(1).

Pregunta de Reflexión

¿Por qué es necesario usar una variable temporal (como actual) en el método recorrer en lugar de mover directamente la variable cabeza?

Uso actual porque si muevo cabeza directamente, pierdo la referencia al inicio de la lista. Los nodos quedarían sin acceso y se perdería toda la lista. Con actual recorro una copia de la referencia y la cabeza se queda intacta.

Nombre: José Alejandro Olvera Lugo Fecha: 15/10/2026

Evaluación: Estructuras Dinámicas en Java

Parte 1: Análisis de Complejidad (Big O)

Compara el rendimiento de los arreglos estáticos frente a las listas enlazadas dinámicas seleccionando la respuesta correcta.

1. ¿Cuál es la complejidad temporal para acceder a un elemento por su índice en una **lista enlazada**?

A) O(1)
B) O(n)
C) O(log n)

2. Al **insertar un elemento al inicio**, ¿qué estructura es más eficiente?

A) El arreglo, porque tiene acceso O(1).
B) La lista enlazada, porque solo requiere reasignar referencias O(1).
C) Ambas tienen la misma eficiencia O(n).

3. ¿Por qué un **arreglo** tiene una complejidad de O(1) para el acceso por índice?

A) Porque utiliza memoria no contigua.
B) Porque el Garbage Collector optimiza la búsqueda.
C) Porque al estar en memoria contigua, se puede calcular la dirección exacta del dato.

4. Si necesitas buscar un dato específico recorriendo todos los elementos anteriores, la complejidad es:

A) O(n)
B) O(1)
C) $O(n^2)$

5. ¿Cuál es una desventaja de la lista enlazada en términos de eficiencia de memoria?

A) La memoria es estática y no se puede liberar.
B) Ocupa menos espacio que un arreglo.
C) Cada nodo consume memoria extra para almacenar la referencia al siguiente elemento.

Parte 2: Gestión de Memoria en Java

Responde Verdadero (V) o Falso (F) a las siguientes afirmaciones sobre el manejo de memoria.

6. En Java, los objetos creados con la palabra new se almacenan en una región de memoria llamada *heap*.

A) Verdadero
B) Falso

7. El *Garbage Collector* libera automáticamente la memoria de los nodos que aún tienen una referencia activa apuntando a ellos.

A) Falso
B) Verdadero

8. Una estructura dinámica se diferencia de una estática porque su tamaño puede crecer o decrecer durante la ejecución.

A) Falso
B) Verdadero

Parte 3: Lógica de Referencias y Fugas de Memoria

Responde de manera detallada a las siguientes preguntas sobre la manipulación de nodos.

9. Explica por qué es crítico conectar el nuevo nodo a la lista **antes** de actualizar la referencia de la 'cabeza' durante una inserción al inicio.

Es crítico conectar primero el nuevo nodo a la lista porque la cabeza es la única referencia que tengo al inicio de la lista. Si actualizo la cabeza antes de que el nuevo nodo apunte a la cabeza anterior, pierdo el acceso a todos los nodos existentes. Al conectar primero (nuevo.siguiente = cabeza) y después mover la cabeza (cabeza = nuevo), me aseguro de que nada quede suelto.

---

10. ¿Qué sucede con la memoria en Java si desconectas accidentalmente el primer nodo de una lista sin haber guardado una referencia al resto de los elementos? Justifica tu respuesta mencionando el concepto de *Garbage Collector*.

Los nodos que quedan sin ninguna referencia apuntándolos son inalcanzables para el programa. Cuando eso pasa, el Garbage Collector los detecta como basura y libera su memoria automáticamente. Entonces se pierde todo el resto de la lista, porque al desconectar el primer nodo sin guardar una referencia al segundo, ya no hay forma de llegar a esos datos.
